

2.10 Spring Web MVC Homework

1. Make a list of all the annotations we have seen so far in the Spring Web Framework.

◆ Spring MVC Annotations

- **@Controller** → Marks a class as a *Spring MVC controller* (used for handling web requests, usually returning views).
- **@ResponseBody** → Directly *serializes* method return values to *JSON/XML* (instead of returning a view).
- **@RestController** → Combines **@Controller** + **@ResponseBody**, making it easier for *REST APIs*.
- **@RequestMapping** → Maps *HTTP requests* to *classes* or *methods* (supports *method, path, params, headers*, etc.).
- **@GetMapping**, **@PostMapping**, **@PutMapping**, **@DeleteMapping**, **@PatchMapping** → Shorthand annotations for **@RequestMapping** with specific *HTTP verbs*.
- **@PathVariable** → Binds *URL path* values to *method parameters*.
- **@RequestParam** → Binds *query parameters* (?id=1) or form data to *method parameters*.

◆ JPA (Persistence) Annotations

- **@Entity** → Marks a *class* as a *JPA entity* mapped to a *database table*.
- **@Repository** → Marks a *DAO* (Data Access Object) as a Spring-managed *bean* with *exception translation*.

◆ Lombok Annotations

- **@Getter / @Setter** → Auto-generates *getter* and *setter methods*.
- **@NoArgsConstructor** → Generates a *no-argument constructor*.
- **@AllArgsConstructor** → Generates a *constructor with all fields*.
- **@RequiredArgsConstructor** → Generates a constructor for *all final* and *@NonNull fields*.
- **@Data** → Generates **@Getter**, **@Setter**, **@ToString**, **@EqualsAndHashCode**, and **@RequiredArgsConstructor**.
- **@Builder** → Implements *Builder pattern* for object creation.

◆ Validation Annotations (Jakarta/Javax Validation)

- **@NotBlank** → Validates that a string is *not null* and *not empty* (ignores *whitespace*).
- **@NotEmpty** → Validates that a collection/string is *not null* and *not empty*.
- **@NotNull** → Validates that a value is *not null*.
- **@Email** → Validates that a string *follows email format*.
- **@NegativeOrZero** → Validates that a *number* is ≤ 0 .
- **@FutureOrPresent** → Validates that a *date/time* is *now* or *in the future*.
- **@AssertTrue** → Validates that a *boolean field* must be *true*.
- **@Valid** → Tells *Spring* to *recursively validate* a *field/object*.
- **@Constraint(Validator.class)** → Custom validation annotation that *points to a class* implementing ConstraintValidator.

◆ Meta Annotations

- **@Retention(RetentionPolicy)** → Defines *how long* an *annotation is retained*:
 - **SOURCE** (discarded after compilation),
 - **CLASS** (in class file but *not at runtime*),
 - **RUNTIME** (available at runtime via reflection).
- **@Target(ElementType)** → Defines *where* the *annotation can be applied* (class, method, field, etc.).

◆ Spring Exception Handling

- **@ExceptionHandler(Exception.class)** → Marks a method to *handle* a *specific exception*.
- **@ControllerAdvice** → Global *interceptor for controllers* (can handle exceptions, bind data, etc.).
- **@RestControllerAdvice** → Combination of **@ControllerAdvice** + **@ResponseBody**, global exception handler for REST APIs.

◆ Jackson (JSON) Annotation

- **@JsonFormat** → Defines *how* a *property* is *serialized/deserialized* (e.g., format a *LocalDate* as "dd-MM yyyy").

2. Create the following REST endpoints for the following Entity

Controller:

DepartmentEntity
<pre>@Entity @Data public class DepartmentEntity { @Id @GeneratedValue(strategy = GenerationType.AUTO) private Long id; private String title; private Boolean active; private LocalDate createdAt; }</pre>

Department	REST APIs:
- id	GET: /departments
- title	POST: /departments
- isActive	PUT: /departments
- createdAt	DELETE: /departments
	GET: /departments/{id}

DepartmentController
<pre>@RestController @RequestMapping("/department") public class DepartmentController { private final DepartmentService service; public DepartmentController(DepartmentService service) { this.service = service; } @GetMapping() public ResponseEntity<List<DepartmentDTO>> getAllDepartments() { List<DepartmentDTO> departments = service.getAllDepartments(); return (departments == null) ? ResponseEntity.noContent().build() : ResponseEntity.ok(departments); } @GetMapping("/{id}") public DepartmentDTO getDepartmentById(@PathVariable("id") Long id) { return service.getDepartmentById(id); } @PostMapping public ResponseEntity<DepartmentDTO> createDepartment(@Valid @RequestBody DepartmentDTO body) { return new ResponseEntity<>(service.createDepartment(body), HttpStatus.CREATED); } @PutMapping("/{id}") public DepartmentDTO updateDepartment(@PathVariable("id") Long id, @Valid @RequestBody DepartmentDTO body) { return service.updateDepartment(id, body); } @DeleteMapping("/{id}") public ResponseEntity<Boolean> deleteDepartmentById(@PathVariable("id") Long id) { return new ResponseEntity<>(service.deleteDepartment(id), HttpStatus.NO_CONTENT); } @PatchMapping("/{id}") public DepartmentDTO updateDepartmentPartially(@PathVariable("id") Long id, @Valid @RequestBody Map<String, Object> updates) { return service.updateDepartmentPartially(id, updates); } }</pre>

@Service

```

public class DepartmentService {
    private final DepartmentRepository repository;
    private final ObjectMapper mapper;
    // constructor injection
    public DepartmentService(DepartmentRepository repository, ObjectMapper mapper) {
        this.repository = repository;
        this.mapper = mapper;
    }
    public List<DepartmentDTO> getAllDepartments() {
        List<DepartmentEntity> departments = repository.findAll();
        return departments.stream()
            .map(entity -> mapper.convertValue(entity, DepartmentDTO.class))
            .toList();
    }
    public DepartmentDTO getDepartmentById(Long id) {
        existById(id);
        DepartmentEntity department = repository.findById(id).get();
        return mapper.convertValue(department, DepartmentDTO.class);
    }
    public DepartmentDTO createDepartment(DepartmentDTO body) {
        DepartmentEntity newDepartment = repository.save(mapper.convertValue(body, DepartmentEntity.class));
        return mapper.convertValue(newDepartment, DepartmentDTO.class);
    }
    public DepartmentDTO updateDepartment(long id, DepartmentDTO body) {
        existById(id);
        body.setId(id);
        DepartmentEntity updatedDepartment = mapper.convertValue(body, DepartmentEntity.class);
        repository.save(updatedDepartment);
        return mapper.convertValue(updatedDepartment, DepartmentDTO.class);
    }
    public Boolean deleteDepartment(long id) {
        existById(id);
        repository.deleteById(id);
        return true;
    }
    public DepartmentDTO updateDepartmentPartially(Long id, Map<String, Object> updates) {
        existById(id);
        DepartmentEntity existingDepartment = repository.findById(id).get();
        DepartmentEntity updatedDepartment = mapper.convertValue(updates, DepartmentEntity.class);
        String[] nullFields = getNullPropertyNames(updatedDepartment);
        BeanUtils.copyProperties(updatedDepartment, existingDepartment, nullFields);
        return mapper.convertValue(repository.save(existingDepartment), DepartmentDTO.class);
    }
    private String[] getNullPropertyNames(Object source) {
        BeanWrapper src = new BeanWrapperImpl(source);
        return Arrays.stream(src.getPropertyDescriptors())
            .map(FeatureDescriptor::getName)
            .filter(name -> src.getPropertyValue(name) == null)
            .toArray(String[]::new);
    }
    private void existById(Long id) {
        if (!repository.existsById(id)) {
            throw new ResourceNotFoundException("Department not exists with id: " + id);
        }
    }
}

```

3. Write Exception Handling logic for the Department Entity.

GlobalExceptionHandler

@RestControllerAdvice

```
public class GlobalExceptionHandler {
    // Handles No-Such-Element-Found exception
    @ExceptionHandler(NoSuchElementException.class)
    public ResponseEntity<ApiResponse<?>> handelNoSuchElementException(NoSuchElementException exception) {
        ApiError error = ApiError.builder()
            .status(HttpStatus.NOT_FOUND)
            .message(errorMessage(exception))
            .errors(List.of(exception.getMessage()))
            .build();

        return buildErrorResponse(error);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiResponse<?>> handelInternalServerError(Exception e) {
        ApiError error = ApiError.builder()
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .message(errorMessage(e))
            .errors(List.of(e.getMessage()))
            .build();

        return buildErrorResponse(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiResponse<?>> handleInputValidationError(MethodArgumentNotValidException e) {
        List<String> errors = new LinkedList<>();
        e.getBindingResult()
            .getAllErrors()
            .forEach(err -> errors.add(err.getDefaultMessage()));
        ApiError error = ApiError.builder()
            .status(HttpStatus.BAD_REQUEST)
            .message(errorMessage(e))
            .errors(errors)
            .build();
        return buildErrorResponse(error);
    }

    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    public ResponseEntity<ApiResponse<?>> handelMethodArgumentTypeMismatchError(MethodArgumentTypeMismatchException e) {
        ApiError error = ApiError.builder()
            .status(HttpStatus.BAD_REQUEST)
            .message(errorMessage(e))
            .errors(List.of(e.getMessage()))
            .build();
        return buildErrorResponse(error);
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiResponse<?>> handleResourceNotFoundError(ResourceNotFoundException e) {
        ApiError error = ApiError.builder()
            .status(HttpStatus.NOT_FOUND)
            .message(errorMessage(e))
            .errors(List.of(e.getMessage()))
            .build();
        return buildErrorResponse(error);
    }

    private ResponseEntity<ApiResponse<?>> buildErrorResponse(ApiError error) {
        return new ResponseEntity<>(new ApiResponse<>(error, "Exception occurred"), error.getStatus());
    }

    private String errorMessage(Exception e) {
        String[] err = e.getClass().getName().split("\\.");
        return err[err.length - 1];
    }
}
```

4. Add the appropriate fields in the Department and Employee Entities so that the following validations can be added:

@Null, @NotNull, @AssertTrue, @AssertFalse, @Min, @Max, @DecimalMin, @DecimalMax, @Negative, @NegativeOrZero, @Positive, @PositiveOrZero, @Size, @Digits, @Past, @PastOrPresent, @Future, @FutureOrPresent, @Pattern, @Email, @NotEmpty, @NotBlank, @Length, @Range, @CreditCardNumber, @URL

Employee entity

```
@Data
public class EmployeeDTO {
    private Long id;
    @NotBlank
    @Size(min = 3, max = 25)
    private String name;
    @Prime(message = "luckyNumber must be a Prime")
    private Integer luckyNumber;
    @Email(message = "Enter valid email")
    private String email;
    @Password(message = "Password is too weak. It must be at least 8 characters long and include uppercase letters, lowercase letters, special characters and numbers")
    private String password;
    @Digits(integer = 6, fraction = 2, message = "Salary must be in format: [XXXXXX.YY]")
    @DecimalMin("9999.99")
    @DecimalMax("200000.50")
    private Double salary;
    @Positive(message = "Enter a valid age")
    @Min(value = 18, message = "Employee must be an adult(at-least 18)")
    private Integer age;
    @AssertTrue(message = "active must be true")
    private Boolean active;
    @RoleValidation(message = "Employee role must be either USER or ADMIN")
    private String role;
    @PastOrPresent
    @JsonFormat(pattern = "yyyy-MM-dd")
    private LocalDate dateOfJoining;
}
```

5. Create custom Annotation to handle a Prime number input.

@Prime

```
@Constraint(validatedBy = PrimeValidator.class)
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.FIELD, ElementType.PARAMETER})
public @interface Prime {
    String message() default "Input integer must be a prime number";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

PrimeValidator

```
@Override
public boolean isValid(Integer number, ConstraintValidatorContext constraintValidatorContext) {
    if (number == null) return false;
    for (int i = 2; i * i <= number; i++) {
        if (number % i == 0) return false;
    }
    return true;
}
```

6. Create custom Annotation to check if the Password String field satisfies the following criteria:

- a. contains one uppercase letter
- b. contains one lowercase letter
- c. contains one special character
- d. minimum length is 10 characters

@Password

```
@Constraint(validatedBy = PasswordValidator.class)
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.FIELD, ElementType.PARAMETER})
public @interface Password {
    String message() default "{jakarta.validation.constraints.NotBlank.message}";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

PasswordValidator

```
public class PasswordValidator implements ConstraintValidator<Password, String> {
    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {
        if (value == null) return false;
        Pattern pattern = Pattern.compile("(?=.*[A-Z])(?=.*[a-z])(?=.*[0-9])(?=.*[!@#-zA-Z0-9]).{10,}$");
        return pattern.matcher(value).matches();
    }
}
```